



ExplainableDetector: Exploring transformer-based language modeling approach for SMS spam detection with explainability analysis [☆]

Mohammad Amaz Uddin ^{a, }, Muhammad Nazrul Islam ^{b, }, Leandros Maglaras ^{c, },
Helge Janicke ^{d, }, Iqbal H. Sarker ^{d, }, ^{*}

^a Department of Computer Science and Engineering, BGC Trust University Bangladesh, Chittagong, 4381, Bangladesh

^b Department of Computer Science and Engineering, Military Institute of Science and Technology, Dhaka, 1216, Bangladesh

^c School of Computing, De Montfort University, UK

^d Centre for Securing Digital Futures, Edith Cowan University, Perth, WA-6027, Australia

ARTICLE INFO

Keywords:

Cybersecurity
Machine learning
Large language model
Spam detection
Text analytics
Explainable AI
Fine-tuning
Transformer

ABSTRACT

Short Message Service (SMS) is a widely used and cost-effective communication medium that has unfortunately become a frequent target for unsolicited messages - commonly known as SMS spam. With the rapid adoption of smartphones and increased Internet connectivity, SMS spam has emerged as a prevalent threat. Spammers have recognized the critical role SMS plays in today's modern communication, making it a prime target for abuse. As cybersecurity threats continue to evolve, the volume of SMS spam has increased substantially in recent years. Moreover, the unstructured format of SMS data creates significant challenges for SMS spam detection, making it more difficult to successfully combat spam attacks. In this paper, we present an optimized and fine-tuned transformer-based Language Model to address the problem of SMS spam detection. We use a benchmark SMS spam dataset to analyze this spam detection model. Additionally, we utilize pre-processing techniques to obtain clean and noise-free data and address class imbalance problem by leveraging text augmentation techniques. The overall experiment showed that our optimized fine-tuned BERT (Bidirectional Encoder Representations from Transformers) variant model RoBERTa obtained high accuracy with 99.84%. To further enhance model transparency, we incorporate Explainable Artificial Intelligence (XAI) techniques that compute positive and negative coefficient scores, offering insight into the model's decision-making process. Additionally, we evaluate the performance of traditional machine learning models as a baseline for comparison. This comprehensive analysis demonstrates the significant impact language models can have on addressing complex text-based challenges within the cybersecurity landscape.

1. Introduction

In recent years, SMS has become the primary communication tool as mobile phones and networks have proliferated [1]. It is quite popular since it's quick and convenient, and it responds to messages faster than emails [2]. Due to its convenience, accessibility, and broad use in our increasingly interconnected society, SMS is now the most popular form of communication for both individuals and organizations. Increasingly, SMS has become a powerful tool for global communication due to its several benefits, such as simplicity of use, cost-effectiveness, wide reach, and more. Additionally, SMS promotes customer relationships, provides company updates, and provides discreet individualized communication,

all of which contribute to real-time audience engagement. It is effectively used in various industries, including marketing, retail, healthcare, and finance. However, because of these benefits, an increasing number of mobile users are utilizing SMS facilities, which encourages spammers to generate spam SMS attacks aimed at harming users [3]. Spammers primarily gather phone numbers via a variety of methods and then utilize automated software services to deliver large volumes of messages to these numbers. Phishing URLs (Uniform Resource Locators), fraudulent schemes, and other fraudulent content are promoted in these bulk messages. Financial gain is their primary objective, which is attained by purchasing counterfeit items, obtaining personal information, or clicking on malicious links that download malware [4]. Although spam detection is

[☆] Peer review under the responsibility of the Chongqing University of Posts and Telecommunications.

^{*} Corresponding author.

E-mail address: m.sarker@ecu.edu.au (I.H. Sarker).

<https://doi.org/10.1016/j.dcan.2025.07.008>

Received 27 May 2024; Received in revised form 2 July 2025; Accepted 14 July 2025

a challenging task in the cybersecurity field, significant enhancements to these efforts can be achieved by leveraging Artificial Intelligence (AI) methods, including Machine Learning, Deep Learning (DL), Hybrid Learning, and Transfer Learning, which have proven effective in enhancing spam detection efforts [5]. Furthermore, due to rapid technological advancements, novel AI-based methods are being developed within the area.

Large Language Models (LLMs) research, supported by both industry and academia, has been essential in the recent advances in natural language interpretation [6]. The field of artificial intelligence research and application could undergo a revolutionary shift because of these models [7]. These language models are trained on expansive datasets and excel in capturing nuanced linguistic patterns. Utilizing the power of LLMs, companies like Google, Microsoft, Amazon, and OpenAI are developing different AI tools that are skilled at understanding and providing natural language responses to user's requests. Notably, OpenAI's ChatGPT and Google's Bard exemplify the transformative power of LLMs, captivating widespread attention for their ability to generate and understand human-like text [8]. For natural language processing applications, transformer-based models such as GPT (Generative Pre-trained Transformer) and Google's BERT (Bidirectional Encoder Representations from Transformers) are referred to as LLMs because of their capacity to understand and generate text that is similar to that of a human. These transformer-based models are useful for a variety of tasks in NLP research and applications because of their versatility in handling translation, generation, and language comprehension.

In the field of NLP, a transformer [9] refers to a deep learning model architecture that is becoming more popular in text classification and translation tasks. Its creative architecture makes use of attention mechanisms to reliably encode input data into representations, providing unmatched performance across a range of natural language processing applications [10]. Transformer-based models have brought in a new era of deep learning capabilities, demonstrating their transformational impact on the discipline in areas ranging from spam categorization to text comprehension and interpretation [11]. Recently, improved transformer-based models, such as BERT, were presented to address several NLP problems. The BERT-language representation model is a transformer-based neural network architecture that has been pre-trained. It is extremely effective and useful for a variety of text-related applications, such as spam and phishing detection [12]. By utilizing bidirectional attention mechanisms, BERT can comprehend the complex relationships between words and phrases within a sentence by grasping contextual word embeddings [13]. The ability of BERT to take into account context from both ends (left and right) of a sentence sets it apart from previous models that were restricted to unidirectional processing [14]. This bidirectional approach gives BERT a better understanding of the language nuances and relationships present in the text. Using a Masked Language Model (MLM) approach, BERT trains the model to infer the original words from context by substituting specific words in a sentence with a unique token ([MASK]). Furthermore, BERT has other variations, including BERT-base, BERT-large [15], ALBERT [16], RoBERTa [17], DistilBERT [18], and TinyBERT [19], among others. These variations vary in the number of transformer layers and parameters, and each one provides unique benefits and enhancements for tasks involving natural language processing.

BERT is useful in a variety of NLP tasks, but due to its complexity, it is sometimes seen as a black box, making it difficult to comprehend how it makes predictions. XAI techniques have become crucial tools for clarifying complex procedures [5,20,21]. Through exploring the model's inner workings, these methods provide insightful explanations for why particular predictions are made. Using XAI methods, researchers can clarify the keywords or characteristics that the model identified as most important for certain activities, which improves clarity and transparency [22]. In this research work, our main aim is to clarify the transparency of the model in the SMS spam classification task.

In summary, our major contributions can be outlined as follows:

- We present an SMS spam detection model leveraging transformer-based language models. To make the model more efficient and impactful on text-based spam data, we optimize and fine-tune the DistilBERT and RoBERTa models.
- We calculate the positive and negative coefficients of the text using XAI techniques to find out the spam messages. This enables the explanation and investigation of the fine-tuned model's interpretability.
- We conduct a comparative analysis of experimental results between traditional models and language models on both balanced and imbalanced datasets. To mitigate the class imbalance issue, we incorporate a text augmentation method, allowing us to assess the performance impact of balanced versus imbalanced data.

The remainder of the paper is organized as follows: Section 2 reviews related works. Section 3 presents a comprehensive explanation of the overall methodology. Section 4 conducts an in-depth analysis of the experimental results. Section 5 examines the transparency of the model using XAI approaches. Furthermore, Section 6 provides a detailed discussion of the findings and results. Finally, Section 7 concludes this paper.

2. Literature review

In the rapidly evolving cybersecurity landscape, combating spam SMS messages has become a major opportunity and challenge due to their sophisticated and increasingly significant nature in today's increasingly interconnected world. Indeed, researchers have created several solutions for dealing with spam SMS by utilizing cutting-edge methods from machine learning, deep learning, and hybrid models. These initiatives highlight how constantly evolving the sector is, with new techniques like LLMs, anomaly detection, pattern recognition, and ensemble methods being employed to identify and counter cyber threats as they arise.

2.1. Machine learning techniques

To prevent security threats, maintain customer engagement, safeguard their reputation, enhance operational efficiency, and comply with regulations, organizations must quickly detect spam attempts. By implementing efficient spam detection systems, businesses can preserve customer confidence and secure communication channels. Many methods utilized in the field of SMS spam classification tasks in the past have contributed to achieving these objectives. Abid et al. [23] presented a method that utilizes supervised machine learning techniques, including TF-IDF (Term Frequency-Inverse Document Frequency) and bag-of-words for feature extraction, to classify spam and ham SMS. To address data imbalance, oversampling and undersampling techniques were implemented. Among several machine learning classifiers tested, Random Forest (RF) achieved a remarkable 99% accuracy in distinguishing between spam and ham SMS. In [24], the authors represented a two-phase smishing (SMS phishing) detection model: the domain checking phase verifies URL authenticity, while the SMS classification phase extracts text features. By using the backpropagation algorithm, the system outperforms traditional classifiers and classifies messages with an accuracy of 97.93%, which demonstrates high efficiency in detecting smishing messages.

The study [25] showed a comprehensive analysis that underscores the effectiveness of feature selection techniques in enhancing smishing detection accuracy. This work investigates feature ranking using four correlation algorithms, such as Pearson rank correlation, Spearman's rank correlation, Kendall rank correlation, and Point biserial rank correlation, alongside a machine learning approach for smishing detection, revealing the AdaBoost classifier as the most accurate. Furthermore, with the Kendall ranking algorithm, rank correlation offered better accuracy than the other correlation algorithms, that is 98.40%. Nagare et

al. [26] investigated the potential use of a machine learning model, such as Naive Bayes (NB), for SMS spam detection using a dataset from the UCI Machine Learning repository. For this context, the dataset is pre-processed and found that the NB model efficiently demonstrated high efficacy on those tests in detecting SMS spam with an impressive classification accuracy of 98.22% for correctly classified samples. Several researchers have proposed a naive bayes approach [27,28] for classifying text messages as spam or ham.

2.2. Deep learning techniques

In the cybersecurity field, deep learning algorithms have become indispensable due to their ability to process complex data structures, allowing them to detect subtle patterns and anomalies. Through the utilization of deep learning, cybersecurity experts can strengthen defense systems against constantly changing dangers, guaranteeing strong safeguarding for critical data and digital assets. In [29], the authors presented a unique method that leverages word order information to address the problem of lower frequency concerns in SMS spam detection: discrete Hidden Markov Models (HMM). The suggested HMM approach showed strong performance equivalent to deep learning techniques like Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) models through experiments on the UCI SMS spam dataset and a Chinese SMS spam dataset. It achieved 95.9% accuracy on the UCI dataset and demonstrates language-agnostic effectiveness in identifying spam SMS. The study [30] suggested using a hybrid deep learning model to identify spam SMS messages in an English and Arabic-supporting mixed mobile environment. CNN and LSTM, two deep learning techniques, are combined in this detection model. Other popular machine learning algorithms were also used to assess the overall procedure, and the results showed that the CNNLSTM model that was suggested performed better than the others, achieving an accuracy of 98.37%.

Roy et al. [31] employed two deep learning techniques together: CNN and LSTM to classify Spam and Not-Spam text messages. They used a spam dataset where 747 spam and 4,827 not-spam text messages exist. To evaluate the performance of the proposed method, the hybrid model was compared with the traditional machine learning models and achieved a comparatively high accuracy of 99.44%. Abayomi-Alli et al. in [32], a deep learning model built on top of Bidirectional Long Short-Term Memory (BiLSTM) is presented, and its effectiveness is evaluated against the latest machine learning techniques on two datasets: the UCI SMS dataset and a recently gathered dataset named ExAIS_SMS. The BiLSTM model outperformed certain ML classifiers, showing a moderate improvement in accuracy over some, with 93.4% accuracy for the ExAIS_SMS dataset and 98.6% accuracy for the UCI dataset, after evaluation using many metrics including True Positive (TP), False Positive (FP), F-measure, recall, precision, and overall accuracy. Further supporting the superiority of the BiLSTM model in spam identification, the comparison also showed significant accuracy differences among ML classifiers.

2.3. Transformer model-based techniques

Transformer-based models like BERT and GPT are very useful for processing and comprehending textual data, and they can also be applied to improve cybersecurity defenses and address the ever-changing cyber threat landscape. The study [33] demonstrated how well BERT performs comprehensive context analysis to enhance spam detection and produce better classification results. Using a pre-trained BERT model, a proposed high-performance spam detector obtains impressive accuracy rates (from 97.83% to 99.28%) on a variety of corpora such as SMS spam collections, Enron, SpamAssassin, and Ling-Spam. For SMS spam identification, Liu et al. [1] introduced both a customized Transformer model and a Vanilla Transformer model with modifications. They utilized the GloVe method for text message vector representations, achieving an accuracy of 98.92%. Using a Kaggle dataset, the authors

Table 1

The sample SMS data overview.

SMS	Class
PRIVATE! Your 2003 Account Statement for 078	spam
I'm in a meeting, call me later at	ham
;-) ok. I feel like john lennon.	ham
I am in office:)whats the matter..msg me now.i will call you at break:).	ham
85233 FREE> Ringtone!Reply REAL	spam
3. You have received your mobile content. Enjoy	spam
Want explicit SEX in 30 secs? Ring 02073162414 now! Costs 20p/min	spam
Gsex POBOX 2667 WC1N 3XX	
Apo all other are mokka players only	ham
Yun ah.now i wkg where?btw if i go nus sc. i wana specialise in wad?	ham
Are you unique enough? Find out from 30th August.	spam
www.areyouunique.co.uk	

[3] introduced a spam detection model that used BERT to outperform both deep learning techniques and conventional machine learning algorithms, with an accuracy of 98.80%. Notably, it optimizes hyperparameters to speed up model selection and improves model performance by training BERT with extra layers on a benchmark dataset.

In [2], the authors addressed dynamic keywords well by concentrating on word semantics in SMS spam filtering through the use of an intention-based approach. Different pre-trained NLP models are used to provide contextual embeddings, and 13 pre-specified intention labels are used to capture the textual and semantic components of short-text messages. By computing intention scores and using supervised learning classifiers, the model obtains impressive results on the SMS Spam Collection benchmark dataset, with an accuracy of 98.07%. The authors [34] highlighted the vital role of self-attention mechanisms in enabling BERT models to detect spam effectively. They developed a Universal Spam Detection Model (USDM) trained on four different publicly available datasets: Enron, Spamassassin, Lingspam, and Spam Text datasets. By fine-tuning with the same hyperparameters and ensuring F1-scores above 0.9 for each dataset when using the corresponding model, this approach achieved an accuracy of 97% with an F1 score of 0.96.

The literature reviews on SMS spam detection tasks highlight existing research works that demonstrate good performance using ML, deep learning, and transformer-based models. However, although those proposed techniques and models work well, but several types of questions can arise, such as “How do the black-box models work?” and “Are the predictions of those black-box models trustworthy or not?”. To answer those questions, XAI has become a prominent field. The XAI approach improves the black-box model’s explainability and transparency so that it can make more interpretable decisions. In this work, alongside developing the detection model, we incorporate XAI techniques to enhance the trustworthiness of the proposed approach.

3. Methodology

This section provides a brief overview of the methodology, outlining the process from data preparation to model decision-making. The overall research methodology is illustrated in Fig. 1.

3.1. Data collection

A publicly available SMS spam dataset has been collected from the UCI machine learning repository for this analysis. This dataset is labeled and collected for mobile phone spam research. The dataset contains 5,574 English SMS messages, all of which are classified as either spam or ham. Table 1 shows a sample of the data. The dataset shows that among 5,574 data points, 4825 data points existed as ham data and 747 as spam. This distribution of data indicates that the dataset has class imbalance issue, which is needed to consider for an effective modeling.

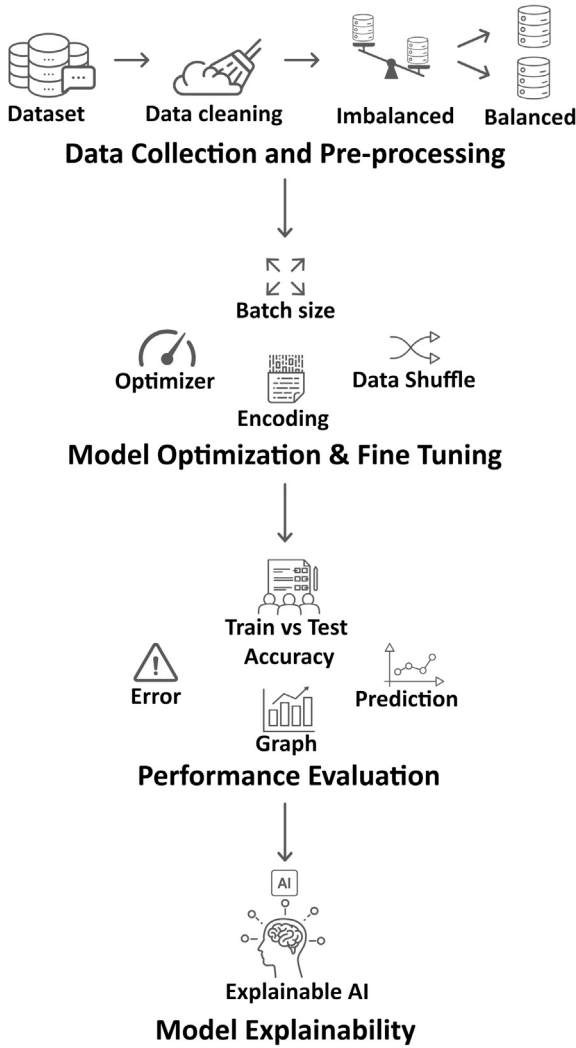


Fig. 1. The methodology of SMS spam detection.

3.2. Data preprocessing

To prepare the dataset, it is essential to clean the data by addressing various issues present within it. We primarily focus on making minimal changes to the data, as excessive pre-processing can potentially reduce the accuracy of NLP tasks. Techniques like punctuation and stopword removal may unintentionally eliminate valuable information from the text. For this spam detection task, we solely clean the data by replacing non-alphanumeric characters in the text with a space. Afterward, we replace all whitespace characters, such as spaces, tabs, and newlines, with a single space. Then, we remove all carriage characters, replacing them with spaces. Finally, we remove all repeated words from the text. To increase the accuracy of the model, pre-processing the SMS data is essential since it highlights important textual characteristics and patterns that are required to recognize spam messages. The Algorithm 1 shows the pre-processing process.

After the data pre-processing phase, we employed a text augmentation technique back translation, and data resampling to balance the data. This augmentation method involves translating the original text data into one or more languages and then translating the translated text back to the original language using machine learning services like Google Translate. Back translation enhances our dataset, making it easier to train robust models, improving performance on a range of natural language processing tasks, and producing text data with different terms while maintaining context. Algorithm 2 shows the process of data aug-

Algorithm 1 Text Preprocessing.

Input: Text to be preprocessed

Output: Cleaned and preprocessed text

```

1: function CLEAN_TEXT(text)
2:   text  $\leftarrow$  REPLACE_SPECIAL_CHARACTERS(text)
3:   text  $\leftarrow$  REMOVE_EXTRA_SPACES(text)
4:   text  $\leftarrow$  REMOVE_CARRIAGE_RETURNS(text)
5:   return text
6: end function
7: function REMOVE_REPEAT(text, repeat)
8:   words  $\leftarrow$  SPLIT_TEXT(text)
9:   result  $\leftarrow$  []
10:  for word in words do
11:    if COUNT_OCCURRENCES(result, word) < repeat then
12:      APPEND(result, word)
13:    end if
14:  end for
15:  return JOIN_WORDS(result)
16: end function
17: function PREPROCESS_TEXT(text)
18:   text  $\leftarrow$  CLEAN_TEXT(text)
19:   text  $\leftarrow$  REMOVE_REPEAT(text, 1)
20:   return text
21: end function

```

mentation using back translation. As in our data, spam is a minor class; we have done overall oversampling techniques in this class. After back translation, we combined the back-translated text with the original text. Overall, through this oversampling process, the dataset has become balanced and eventually the dataset has 4,825 ham and 4,825 spam data in equal portions. Unbalanced datasets can produce biased models that perform poorly in minority classes by favoring the majority class. Balancing the data helps the model learn meaningful features across all classes and enhances its generalization, thereby reducing the risk of overfitting. Fig. 2 illustrates the distribution of imbalanced and balanced data before and after oversampling.

Algorithm 2 Data Augmentation using Back Translation.

Input: Original data

Output: Augmented data using back translation

```

1: function BACK_TRANSLATE(sentence, languages)
2:   translator  $\leftarrow$  INITIALIZE_TRANSLATOR
3:   target_language  $\leftarrow$  RANDOM_CHOICE(languages)
4:   translation  $\leftarrow$  TRANSLATE(translator, sentence, target_language)
5:   translated_text  $\leftarrow$  GET_TRANSLATED_TEXT(translation)
6:   reverse_translation  $\leftarrow$  TRANSLATE(translator, translated_text, 'en')
7:   back_translated_text  $\leftarrow$  GET_TRANSLATED_TEXT(reverse_translation)
8:   return back_translated_text
9: end function
10: for sentence in original_data_list do
11:   augmented_sentence  $\leftarrow$  BACK_TRANSLATE(sentence, [language1, language2, ..., languageN])
12: end for

```

3.3. Dataset splitting

Partitioning a dataset into several subsets is known as dataset splitting, and it is usually done to train and evaluate deep learning or machine learning models. This split is crucial to assess the model's performance on unobserved data. We may determine how effectively a model generalizes to new instances by training it on one subset and testing it on another. This helps to prevent overfitting, a phenomenon in which a model becomes adept at memorizing training data but has trouble with new data. The dataset has been divided into two parts for this experiment: the training set, which contains 80% of the data, and the testing set, which includes the remaining 20%. There are 7,720 data points in the training set and 1,930 data points in the testing set.

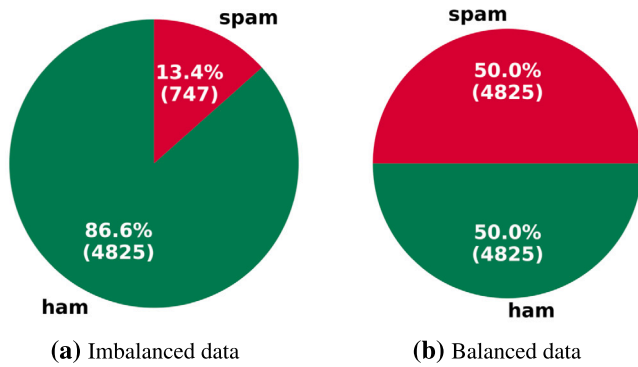


Fig. 2. Visualizing Imbalanced vs. Balanced Data Distribution.

3.4. Model selection

To ensure optimal performance, generalization, and real-world applicability, proper model selection is essential to align the model with the specific requirements, constraints, and characteristics of the classification or detection task. For this detection purpose, initially, we have utilized several traditional ML algorithms named Extreme Gradient Boosting (XGB) [35], Support Vector Machine (SVM) [36], K-Nearest Neighbor (KNN) [37], and Random Forest [38]. Along with the ML models, we also employed transformer-based BERT variant models such as DistilBERT and RoBERTa. The main goal of utilizing both ML models and transformer-based models is to classify SMS spam with high accuracy and to show the best model that performed well in this detection context.

3.4.1. DistilBERT

DistilBERT is a simplified and efficient version of the BERT model. It is designed to provide comparable performance to BERT but at a much smaller, lighter, and faster pace. As a result, it is better suited for implementation in contexts with limited resources or in applications that demand faster inference. DistilBERT is still the preferred transformer-based model for many natural language processing applications because of its effectiveness. It uses a process called knowledge distillation to achieve this. It gains the ability to mimic the functionality of a larger model like BERT using a smaller dataset during training. DistilBERT significantly reduces the size and computing needs of BERT while preserving most of its performance by extracting key insights from the larger model. As a result, it reduces the model size by 40% while retaining 97% of BERT's language understanding ability and achieving a 60% speed improvement [18]. It uses the Transformer architecture, which has feed-forward neural networks similar to BERT and self-attention layers. However, it usually has fewer hidden sizes and fewer layers, resulting in a reduction in the number of parameters. In contrast, the BERT base model typically comprises 12 layers, while DistilBERT may have only 6 layers with a hidden size of 768 [39]. There are about 66 million parameters in DistilBERT, compared to 110 million in the base BERT model. DistilBERT uses tokenization, which turns raw text inputs into numerical embeddings, to handle texts. By utilizing attention mechanisms, these embeddings capture word relationships and help the model identify and prioritize important data.

3.4.2. RoBERTa

Robustly Optimized BERT Approach, or RoBERTa, is an enhanced variant of BERT, a transformer-based model that incorporates improvements to better handle larger batch sizes and train on longer sequences. Similar to BERT, RoBERTa prioritizes the encoder within the Transformer architecture for text and sentence pair classification tasks. Tokenization approach is one of the main differences between these two: BERT uses the WordPiece tokenizer, whereas RoBERTa uses a byte-level Byte Pair Encoding (BPE) tokenizer, employing a larger vocabulary set

comprising 50K subword units [40]. Tokenization is also performed by RoBERTa using BPE as a component rather than character processing directly. RoBERTa learns unsupervised from a large corpus of text data during its pre-training phase. Similar to BERT's Masked Language Model aim, it trains on predicting masked words within sentences. Additionally, to enhance the caliber of learned representations, it does away with the NSP (Next Sentence Prediction) task and employs a larger training corpus with dynamic masking. RoBERTa performs better than BERT, in part, because of this change in training objectives. Its improved performance is also a result of training methodology optimizations, which include bigger batch sizes, longer training times, and dynamic masking.

3.5. Model optimization and fine tuning

Model optimization aims to reduce complexity and increase efficiency, whereas fine-tuning modifies current models to fit new datasets or tasks, improving performance via transfer learning. Both procedures are essential for improving transformer-based models so they perform well in a variety of scenarios. The Transformers tokenizer is used to tokenize the preprocessed spam dataset. It segments text into smaller units using a sub-word-based technique called WordPiece or Byte Pair Encoding, which helps the model better understand word meaning and context. In this tokenization procedure, we initialized the maximum sequence length of the text to 512. Tokenization relies on two key approaches: padding and truncation are essential techniques used to ensure that input sequences have a consistent length. This is particularly important when working with neural networks, as they typically require fixed-size inputs. Padding involves adding tokens (typically zeros) to sequences that are shorter than the maximum length, whereas truncation involves removing tokens from longer sequences.

The choice of optimal batch size is crucial in deep learning, impacting both training and testing accuracies as well as overall runtime [41–43]. This selection involves a trade-off between training speed and memory usage. In this study, a batch size is selected to be 32 for training data, while a batch size of 64 is used for testing data. In order to avoid overfitting and ensure exposure to a wide range of previously unseen samples, the training data is shuffled at each epoch. The model's capacity for generalization is increased and its ability to generalize is strengthened by this randomization. Additionally, pre-trained weights from the pre-training phase initialize the DistilBERT and RoBERTa models, optimizing performance.

An optimizer is an essential part of neural networks that updates pre-trained model weights. An optimizer's primary objective is to minimize a loss function by changing the model's parameters during training. This experiment uses AdamW (Adam Weight Decay) [44], an effective and popular optimization approach, to update the pre-trained model's weights. The AdamW optimizer is a modified and improved version of Adam that is widely used for deep neural network training. It decouples the regularizer's gradient from the Adam- ℓ_2 update rule, making it effective at minimizing a loss function and lowering overfitting [45]. Weight decay, often called L2 regularization, is a regularization technique that is commonly used in neural network training to reduce overfitting. It entails adding a penalty term to the loss function to prevent the use of too high parameter values.

The learning rate is an essential hyperparameter that significantly impacts the dynamics of training and model performance. It determines the size of the steps in the optimization process, which affects convergence and overall efficacy. The ideal learning rate selection is contingent upon several elements, such as the specific task at hand, model design, and optimization techniques. A high learning rate might cause instability, which makes it difficult to generalize to new data. Conversely, a low learning rate could make training take longer, costing more computational resources since more epochs would be needed to reach the target performance levels. For training the DistilBERT and RoBERTa models in our work, we employ a learning rate of $2e-5$ (0.00002), which is a typically used value for transformer-based architectures such as BERT and

Table 2
Hyper-parameter Settings for SMS spam Detection.

Parameter	Utilized Values	Optimal Value
Training Batch size	8, 16, 32, 64	32
Testing Batch size	8, 16, 32, 64	64
Optimizer	AdamW	AdamW
Learning rate	1e-5, 2e-5, 3e-5, 4e-5	2e-5
Number of epochs	2, 3, 4, 5, 6, 7	5

its derivatives. By balancing the trade-off between model performance and training speed, this decision ensures effective convergence while lowering the possibility of instability. For model optimization and fine-tuning the hyperparameter settings for the suggested transformer-based models are shown in Table 2.

After optimizing the model through hyperparameter tuning, the fine-tuning process is implemented to train the model on specific tasks and datasets. This phase involves critical hyperparameters, including batch size, regularization methods, and learning rate. In fine-tuning, an optimization algorithm and loss function are selected, and backpropagation is used to iteratively alter the parameters of the entire model or selected layers. The optimization process modifies the parameters to reduce the difference between the model's predictions and the dataset's ground truth labels. In this experiment, 80% of the entire data was used for fine-tuning. The training data is sent through the models in batches throughout each epoch, and gradients are calculated via the backpropagation technique. To modify pre-trained models for the given task such as SMS spam detection, RobertaForSequenceClassification and DistilBERTForSequenceClassification were utilized. The train_loader and test_loader were configured using DataLoader. The train_loader was configured by preparing the data, figuring out the batch size, and reorganizing the data among epochs. However, no shuffling was involved in configuring the test loader. The overall fine-tuning process flow diagram, featuring these procedures for DistilBERT and RoBERTa, is displayed in Fig. 3.

After setting up the optimizer with its parameters and learning rate, the model receives input parameters during the forward pass to generate predictions. Then, using automatic differentiation, the gradients of the loss with respect to the model's parameters such as weights and biases, are calculated in the backward pass. This determines the loss function's gradients in relation to each parameter. The optimizer uses these gradients to update the model's parameters during the optimization process. This overall process iterates until all the epochs of the model training come to an end. As a result, the model can recognize important patterns and features in the training set. Once the training is complete, the fine-tuned model is ready to predict the spam messages with unseen new data. Algorithm 3 shows the fine-tuning steps of the optimized transformer-based model with the training data.

3.6. Model explainability

Explainability of a model refers to the ability to comprehend and analyze the process by which a model makes its predictions or decisions. A model can be very accurate across various tasks, but it often behaves like a black box, making it challenging to understand why it predicts specific outcomes. Approaches for elucidating black-box machine learning models aim to enhance transparency, yet their explanations may harbor significant ambiguity, leading to diminished user confidence and raising concerns regarding the model's fairness and reliability [46]. XAI aims to develop machine learning methods that enable people to understand and trust the judgments made by complex models, especially deep neural networks, which frequently operate as opaque black boxes [47]. The necessity for greater openness in these models may impede the adoption and trustworthiness of AI and ML technologies. This issue is addressed by XAI, which offers techniques for interpreting and explaining model predictions. An in-depth understanding of XAI's fundamental

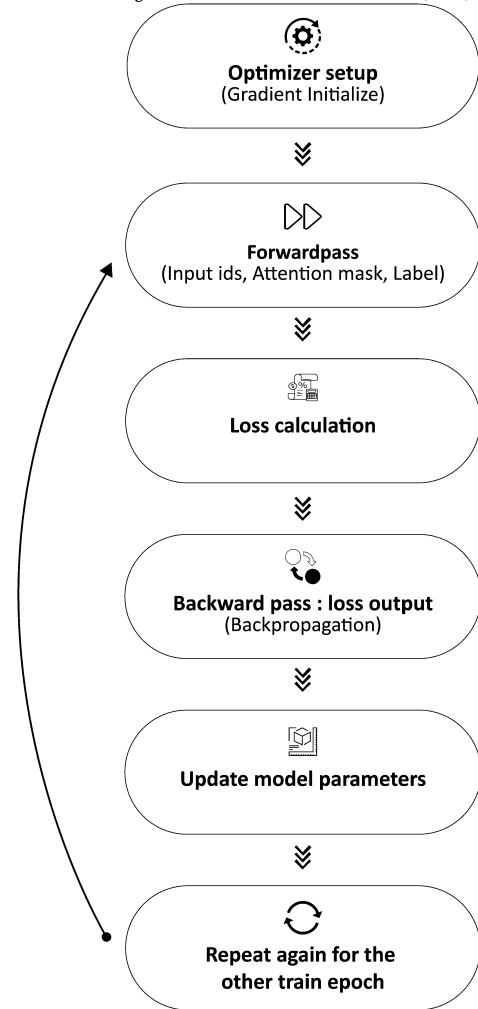


Fig. 3. The Complete Fine-Tuning Process Flow.

concepts and familiarity with related programming frameworks are essential for selecting the appropriate methodology when developing an application that leverages XAI [48]. The XAI community has developed numerous effective methods for explaining predictions, which is crucial for fostering acceptance and confidence in AI and ML technologies. In this research, we utilized two Explainable Models, LIME and Transformers Interpret, to understand the model predictions and address the black box problem. The comprehensive process of model explainability is illustrated in Fig. 4.

The experiment for SMS spam detection is a text classification challenge in which LimeTextExplainer is used to explain how the black box transformer-based model operates. It is an interpretable explanation approach that builds a local interpretable model around a prediction to make any classifier's predictions more understandable [49]. To achieve model explainability, LIME initiates the process by perturbing the SMS content through subtle modifications such as rearranging or removing words or adding new ones. This perturbation investigates how sensitive the model is to changes in the input data. Then, the modified SMS content is fed into the transformer-based model to produce prediction results. LIME extracts prominent characteristics from the resulting instances that are affected by the perturbations. These local models simulate how the black-box model would behave close to the chosen instance and offer insights into how it makes decisions. After that, LIME uses the knowledge gathered from the local model to explain the predictions made by the black-box model on the original dataset. These justifications usually highlight the terms or critical components that substantially impact the model's decision-making process, improving the

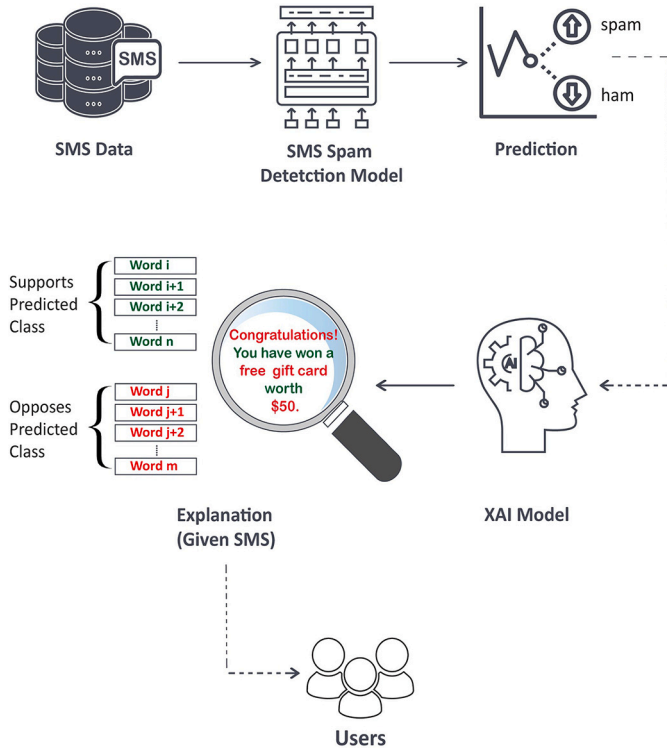
Algorithm 3 Fine-Tuning Transformer-based Model.**Input:** Pre-trained model, train_loader, optimizer, learning rate, maximum training epochs**Output:** Fine-tuned Transformer-based model

```

1: epochs ← max_epochs
2: optimizer ← Optimizer(model.parameters(), learning_rate)
3: device ← cuda
4: function FINETUNETRANSFORMERMODEL(model, train_loader, optimizer, epochs)
5:   for epoch in range(epochs) do
6:     model.train()
7:     for batch in train_loader do
8:       batch_input_ids ← batch[0].to(device)
9:       batch_attention_mask ← batch[1].to(device)
10:      batch_labels ← batch[2].to(device)
11:      optimizer.zero_grad()
12:      outputs ← model(input_ids ← batch_input_ids, attention_mask ← batch_attention_mask, labels ← batch_labels)
13:      calculated_loss ← outputs.loss
14:      calculated_loss.backward()
15:      optimizer.step()
16:   end for
17: end for
18: return model
19: end function

```

▷ Clear gradients
 ▷ Forward pass
 ▷ Backward pass
 ▷ Update parameters

**Fig. 4.** The complete approach to understanding model explainability.

predictive model's overall interpretability. By taking a thorough and iterative approach, LIME helps to clarify the complicated inner workings of machine learning models, increasing transparency and confidence in their forecasts. Eq. (1) shows the formulation used by LIME to generate an interpretable explanation for a specific data point [50].

$$LIME_exp(n) = \underset{x \in X}{\operatorname{argmin}} (L(m, x, \pi_n) + \Omega(x)) \quad (1)$$

In this context, m denotes the black box model that predicts results based on input data n . The LIME model aims to generate an understandable explanation for a particular data point n , denoted as x . An interpretable model x approximates the behavior of the black-box model m locally around n , resulting in the explanation. The hypothesis space of a specific interpretable learner is represented by X . $L(m, x, \pi_n)$ measures the average loss between the predictions of the interpretable model x

and the black-box model m for data points in the neighborhood of π_n . In the context of local approximation, π_n represents the perturbations or features around n that are considered. These perturbations aid in comprehending the behavior of the black-box model near n . $\Omega(x)$ represents a regularization term or penalty applied to the complexity of the interpretable model x . This term penalizes overly complicated models, encouraging simpler explanations.

In order to clarify the operation of the transformer-based model, a Transformers Interpret tool was also included in the categorization process. This tool uses a class explainer technique that uses a tokenizer and pre-trained model to help explain model predictions. Word attributions are computed as part of the class explainer process to reveal how individual words contribute to the model's predictions. The terms that significantly influence the model's decision-making are identified using word attribution analysis. Positive scores represent a word's influence that is positive on the predicted class, and negative scores represent a negative term. In addition, this procedure's explanation highlights terms that significantly impact the model's decision-making. The words that have been highlighted function as crucial identifiers of the attributes that the model gives priority to during the prediction process. The procedure for creating explanations for model predictions using this class explainer approach is described in detail in Algorithm 4. By using this technique, users can enhance transparency and trust in their predictions, gaining a deep understanding of the inner workings of transformer-based models.

4. Result analysis

In this section, we describe the proposed method's comprehensive outcomes and provide a proper comparison between traditional ML models (XGB, SVM, KNN, and RF) utilized in this work. The experiment uses several key metrics such as precision, recall, F1-score, accuracy, and AUC (Area Under the Curve) to analyze the results of a classification model. Our utilized dataset is significantly imbalanced as mentioned earlier. In this experiment, we utilized both imbalanced and balanced dataset to analyze the model behavior. To evaluate the performance of the fine-tuned RoBERTa model on imbalanced and balanced data popular key metrics have been calculated, discussed below.

To summarize the overall performance of a model, calculating the confusion matrix is essential as it forms the foundation for all subsequent key metrics that are derived directly from it. It is a 2x2 table or performance measuring tool that overviews how effectively a classification model has made predictions. This confusion matrix consists

Algorithm 4 Explainability Analysis using XAI.**Input:** Model, tokenizer, input text**Output:** Explanation with word attributions for model predictions

```

1: class_names ← [Class1, Class2, Class3, ..., ClassN]
2: function EXPLAIN_MODEL_PREDICTION(model, tokenizer, input_text)
3:   lime_exp ← EXPLAINWITHLIME(model, input_text, num_features, top_labels)
4:   transformer_exp ← TRANSFORMER_INTERPRETATION(model, tokenizer, input_text)
5:   return lime_exp, transformer_exp
6: end function
7: function EXPLAINWITHLIME(model, input_text, num_features, top_labels)
8:   lime_explainer ← LimeTextExplainer(class_names)
9:   lime_explanation ← lime_explainer.explain_instance(input_text, model, num_features, top_labels)
10:  label ← lime_explanation.get_label()
11:  explanation_attributions ← explanation.as_list(label)
12:  visualize_explanation()
13:  return explanation_attributions
14: end function
15: function TRANSFORMER_INTERPRETATION(model, tokenizer, input_text)
16:  trans_explainer ← SequenceClassificationExplainer(model, tokenizer)
17:  attributions ← trans_explainer(input_text)
18:  visualize_interpretation()
19:  return attributions
20: end function

```

of four entries: True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). A true positive (TP) in a model's evaluation happens when the model accurately predicts a positive outcome and the actual class is positive, proving that positive examples can be accurately identified. Cases where the model accurately predicts a negative outcome and the actual class is negative are referred to be true negative (TN), demonstrating accurate identification of negative events. When the model predicts a positive outcome when the actual class is negative, this is known as a false alarm, or false positive (FP). Lastly, a false negative (FN) occurs when a positive class is observed despite the model predicting a negative outcome. These components of the confusion matrix are the main elements that are used to calculate the Precision, Recall, F1-score, true positive rate (TPR), true negative rate (TNR), false positive rate (FPR), and false negative rate (FNR) that help evaluate the model's performance. Along with that, the ROC (Receiver Operating Characteristic) curve is also generated to compute the model's performance, which is plotted with TRP and FPR.

Precision: Precision, also known as positive predictive value, is the ratio of accurately anticipated positive instances to the total number of predicted positive instances. It measures the model's accuracy in predicting positive outcomes and indicates the likelihood that a predicted positive occurrence is indeed positive. The precision is defined as follows:

$$Precision(P) = \frac{TP}{TP + FP} \quad (2)$$

Recall: Recall, also known as sensitivity, quantifies the proportion of accurately anticipated positive cases to the total actual positive cases in the dataset. It measures the model's ability to detect positive samples and estimates the percentage of real positive cases that the model correctly identifies. The recall is defined as follows:

$$Recall(R) = \frac{TP}{TP + FN} \quad (3)$$

F1-score: The F1-score is a well-rounded performance metric since it uses the harmonic mean to balance recall and precision. A low precision or recall will result in a lower F1-score. It offers a comprehensive assessment of a model's ability to accurately classify positive events by considering the risks of both false positives and false negatives. The F1-Score is defined as follows:

$$F1_score = 2 \times \frac{P \times R}{P + R} \quad (4)$$

Accuracy: Accuracy measures how frequently the model's predictions agree with the actual labels in the dataset. It is computed by dividing the sum of true positives and true negatives by the total number of samples in the dataset. Although accuracy sheds light on the model's overall performance, an uneven class distribution may cause it to underrepresent performance. The accuracy is defined as follows:

$$Accuracy = \frac{TN + TP}{TN + TP + FN + FP} \quad (5)$$

TPR: Measures how well the model correctly recognizes spam SMS. The TPR is defined as follows:

$$TPR = \frac{TP}{TP + FN} \quad (6)$$

TNR: Measures how well the model correctly recognizes ham SMS. The TNR is defined as follows:

$$TNR = \frac{TN}{TN + FP} \quad (7)$$

FPR: Measures how the model incorrectly recognizes spam SMS. The FPR is defined as follows:

$$FPR = \frac{FP}{FP + TN} \quad (8)$$

FNR: Measures how the model incorrectly recognizes ham SMS. The FNR is defined as follows:

$$FNR = \frac{FN}{FN + TP} \quad (9)$$

4.1. Result analysis using imbalanced dataset

The performance of a model can vary based on different parameters. Initially, this study used an imbalanced dataset to evaluate the model's performance during training and testing. The dataset was significantly imbalanced, with a much higher number of ham SMS than spam SMS. Classifiers such as traditional machine learning and transformer-based models were used to classify the text messages into spam and ham classes. The models were trained on the imbalanced dataset using 80% (4,457) of the data and tested on 20% (1,115) of the data. The results obtained from the ML and transformer-based models were impressive for the ham class; however, they were less successful for the spam class due to a lack of data. Table 3 displays the confusion matrix of the traditional ML and transformer-based models for the imbalanced dataset. Comparatively, among the ML models, XGB performed better than SVM, KNN, and RF. While XGB performed well, the RoBERTa model outperformed the ML and transformer-based models, providing the best overall performance. The ML and transformer models' F1-score, recall, and precision were evaluated for the imbalanced dataset, as shown in Table 4. The RoBERTa classifier produced precision, recall, and F1-score values of 0.99, 1.00, and 1.00 for the ham class. On the other hand, the model obtained precision, recall, and F1-score values for the spam class of 0.99, 0.97, and 0.98, respectively. To achieve good accuracy, we trained our optimized and fine-tuned RoBERTa model using 5 epochs with a batch size of 32. After 5 epochs, the model achieved a training accuracy of 99.69% with a training loss of 0.0108. For the test set, it achieved a testing accuracy of 99.37% with a testing loss of 0.0292, using a batch size of 64. Fig. 5 presents a graph showing the training and testing accuracy and loss over 5 epochs for the imbalanced dataset. Fig. 6 and Table 5 illustrate the performance comparison among the algorithms. To provide more assessments such as deeper insight into RoBERTa model performance TPR, TNR, FPR, and FNR are also calculated and its value respectively 0.9667, 0.9979, 0.0021, and 0.0333. The visualization of the ROC curve using TPR and FPR is represented in Fig. 7.

Table 3
Confusion Matrix of ML and Transformer-based Models for Imbalanced Dataset.

Class Name	ML Models								Transformer-based Models			
	XGB		SVM		KNN		RF		DistilBERT		RoBERTa	
	ham	spam	ham	spam	ham	spam	ham	spam	ham	spam	ham	spam
ham	963	2	963	2	665	0	965	0	961	4	963	2
spam	20	130	22	128	98	52	25	125	4	146	5	145

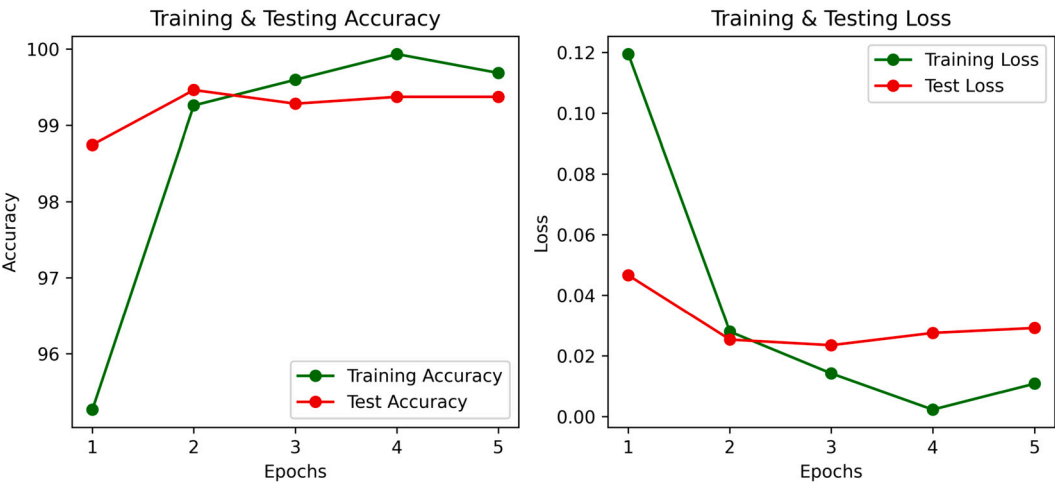


Fig. 5. Training & Testing Accuracy and Loss with Imbalanced Data.

Table 4
Precision, Recall, and F1-Score of ML and Transformer-based Models for Imbalanced Dataset.

Model	Class	Precision	Recall	F1-score
XGB	ham	0.98	1.00	0.99
	spam	0.98	0.87	0.92
SVM	ham	0.98	1.00	0.99
	spam	0.98	0.85	0.91
KNN	ham	0.91	1.00	0.95
	spam	1.00	0.35	0.51
RF	ham	0.97	1.00	0.99
	spam	1.00	0.83	0.91
DistilBERT	ham	1.00	1.00	1.00
	spam	0.97	0.97	0.97
RoBERTa	ham	0.99	1.00	1.00
	spam	0.99	0.97	0.98

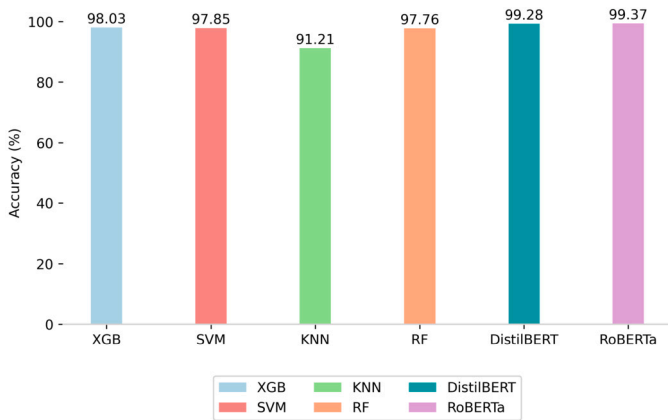


Fig. 6. Representation of graphical comparison among models on an imbalanced dataset.

Table 5
Comparative Performance Analysis of Fine-tuned RoBERTa Model with Other Models on Imbalanced Data.

Model	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
XGB	0.0095	99.84	0.0810	98.03
SVM	0.0077	99.62	0.0591	97.85
KNN	0.1193	91.88	0.5429	91.21
RF	0.0199	100	0.1317	97.76
DistilBERT	0.0034	99.96	0.0343	99.28
RoBERTa	0.0108	99.69	0.0292	99.37

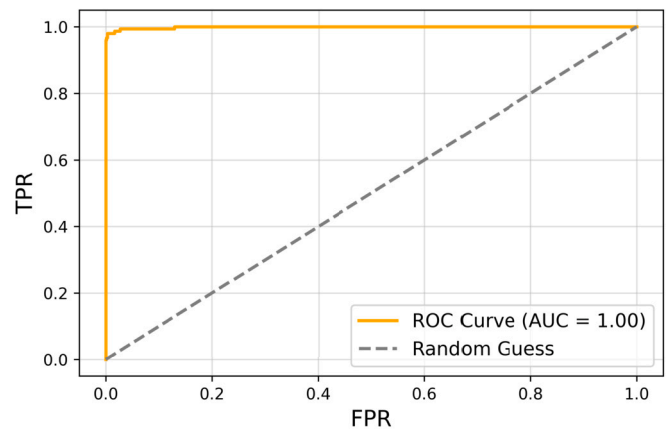


Fig. 7. The ROC curve for RoBERTa on imbalanced data.

4.2. Result analysis using balanced dataset

We also experimented using a balanced dataset. Similar to the imbalanced dataset, we used 80% (7,720 samples) of the data for training and 20% (1,930 samples) for testing. The results from the ML and transformer-based models were impressive for both ham and spam

Table 6

Confusion Matrix of ML and Transformer-based Models for Balanced Dataset.

Class Name	ML Models								Transformer-based Models			
	XGB		SVM		KNN		RF		DistilBERT		RoBERTa	
	ham	spam	ham	spam	ham	spam	ham	spam	ham	spam	ham	spam
ham	932	13	941	4	944	1	943	2	943	2	945	0
spam	19	966	4	981	96	889	10	975	4	981	3	982

Table 7

Precision, Recall, and F1-Score of ML and Transformer-based Models for Balanced Dataset.

Model	Class	Precision	Recall	F1-score
XGB	ham	0.98	0.99	0.98
	spam	0.99	0.98	0.98
SVM	ham	1.00	1.00	1.00
	spam	1.00	1.00	1.00
KNN	ham	0.91	1.00	0.95
	spam	1.00	0.90	0.95
RF	ham	0.99	1.00	0.99
	spam	1.00	0.99	0.99
DistilBERT	ham	1.00	1.00	1.00
	spam	1.00	1.00	1.00
RoBERTa	ham	1.00	1.00	1.00
	spam	1.00	1.00	1.00

Table 8

Comparative Performance Analysis of Fine-tuned RoBERTa Model with Other Models on Balanced Data.

Model	Training Loss	Training Accuracy	Testing Loss	Testing Accuracy
XGB	0.0325	99.72	0.0621	98.34
SVM	0.0041	99.86	0.0323	99.59
KNN	0.0457	97.76	0.1371	94.97
RF	0.0167	100	0.0530	99.38
DistilBERT	0.0015	99.97	0.0150	99.69
RoBERTa	0.0035	99.95	0.0122	99.84

classes. Table 6 shows the confusion matrix of the traditional ML and transformer-based models for the balanced dataset. In this balanced dataset analysis, the SVM algorithm performed notably well among the ML algorithms. Similarly, again our fine-tuned RoBERTa model excelled among the ML and transformer-based models. Table 7 presents the F1-score, recall, and precision evaluation results for the ML and transformer-based models on the balanced dataset. For the ham class, the model achieved precision, recall, and F1-score values of 1.00, 1.00, and 1.00, respectively. For the spam class, the RoBERTa classifier generated F1-score, recall, and precision values of 1.00, 1.00, and 1.00, respectively. With the balanced data, the fine-tuned model achieved 99.95% training accuracy with a training loss of 0.0035 at the 5th epoch. Subsequently, testing the trained model resulted in a testing accuracy of 99.84% and a testing loss of 0.0122. This demonstrates that the model's performance with a balanced dataset is significantly better than with an imbalanced dataset. Fig. 8 displays a graph showing the balanced dataset's accuracy and loss over 5 epochs. Fig. 9 and Table 8 illustrate the performance comparison among the algorithms. The TPR, TNR, FPR, and FNR values measured for the outperformed RoBERTa model respectively showed 0.9970, 1.0000, 0.0000, and 0.003. Fig. 10 visualizes the ROC curve for the RoBERTa model on balanced data which also shows the evidence of effectiveness in this spam classification task.

We have compared our SMS spam detection study with literature studies that worked with similar elements in the same field of cybersecurity. The comparison, summarized in Table 9, evaluates our work from several angles, including the datasets used, the machine and deep learning models used, and whether the study offers decision explainability and human interpretation for the algorithms.

Our transparent research makes it stand out in the field of SMS spam detection. We have provided a detailed explanation of our suggested strategy, which performed better than the existing research works. Interestingly, none of the publications in Table 9 go as deep into a thorough human interpretation of the classifier's output as we have. They do not clarify the model's operation in spam detection.

5. Explainability analysis

This study employs LIME and Transformers Interpret techniques to explain the predictions made by an improved RoBERTa model being used for spam message detection. These tools approximate the model's behavior by analyzing each word's contribution in the SMS text toward the final prediction. LIME and Transformers Interpret aim to answer "Why the model made a particular prediction?". They achieve this by creating word attributions, which give each word in the text a positive or negative coefficient. This helps determine whether terms are more important for categorizing an SMS as spam or ham by showing the impact of each word on the model's prediction. Positive coefficients indicate that the presence of the word contributes towards predicting a certain class, while negative coefficients indicate the opposite. The explainable model highlights words that scored positively or negatively for the prediction, which uses this information to visualize the full text. This visual aid facilitates comprehension of the rationale underlying the model's decision-making process by illuminating key terms that contributed significantly to the categorization result.

In LIME, the corresponding predictions from the complicated model are fitted alongside an interpretable model to the perturbed samples. This procedure is essential to the LIME process to produce better interpretable explanations of the complex model's predictions. This interpretable model's coefficients show how much each feature (word) contributed to creating the predictions. Sometimes, LIME will normalize the data to ensure that the importance rankings are consistent across different models or explanations. In an effort to reduce dimensionality, the study set the initial number of features (words) in the LIME explainer to 15 to accelerate the computational explanation process. Limiting the characteristics simplifies the computational explanation process and provides a clearer, more manageable list of words that have a major influence on the model's prediction for a particular text instance. Fig. 11 and Fig. 12 illustrate the explainability of the RoBERTa model using LIME for both spam and ham, while Table 10, Table 11, Table 12 and Table 13 presents the positive and negative coefficients associated with terms to aid users in understanding spam detection.

We also analyzed Transformers Interpret to assess the model's interpretability with similar SMS texts. This Transformers Interpret illustrates how the transformer-based RoBERTa model uses the tokenizer to divide the huge words into subwords with the prefix "##". This occurs when the model encounters complex or out-of-vocabulary words, which it then breaks down into smaller sub-words to simplify processing. The model can manage a broader vocabulary and comprehend the context of complicated words more accurately due to this procedure. The Transformers Interpret tool's interpretation findings for SMS spam messages are shown in Fig. 13 and Fig. 14, and Table 14, Table 15, Table 16 and Table 17 show the relevance of the words in four SMS, with positive and negative coefficients showing each word's influence on the model's prediction. In this experiment, we worked with a binary classification task

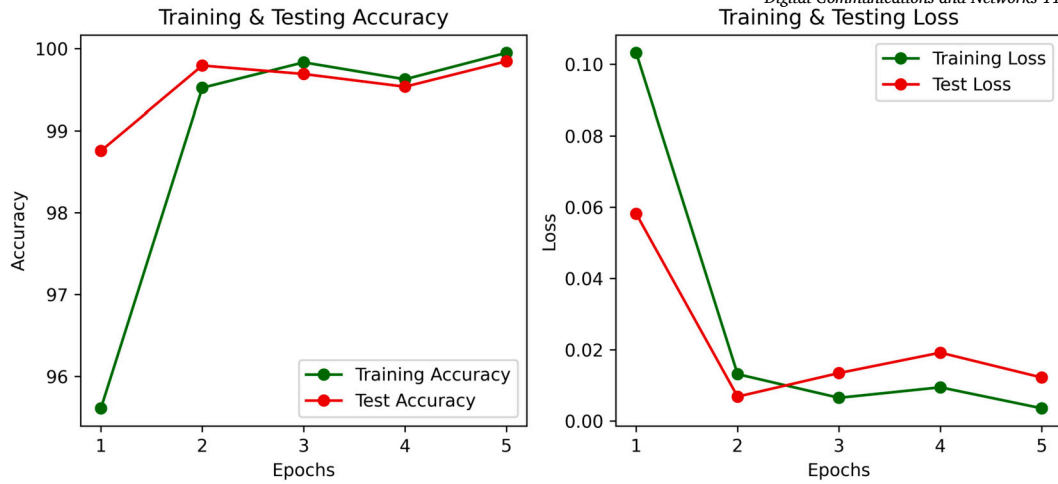


Fig. 8. Training & Testing Accuracy and Loss on Balanced Data.

Table 9

Comparison of Our Study with Various Studies on SMS Spam Detection.

Reference	Datasets	Models	Accuracy (%)	Explainability Analysis
[26]	UCI SMS spam collection	Naive Bayes	98.22	No
[29]	UCI SMS spam collection	HMM	95.9	No
[31]	UCI SMS spam collection	CNN	99.44	No
[3]	UCI SMS spam collection	BERT	98.80	No
[2]	UCI SMS spam collection	DistilBERT+SVM	98.07	No
This Study	UCI SMS spam collection	Fine-tuned RoBERTa	99.84	Yes

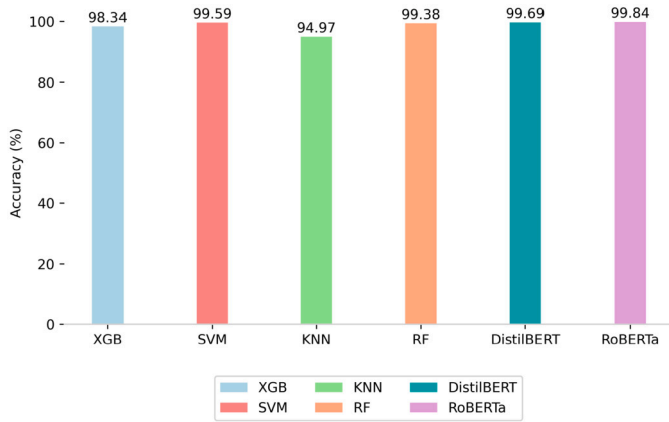


Fig. 9. Representation of graphical comparison among models on balanced dataset.

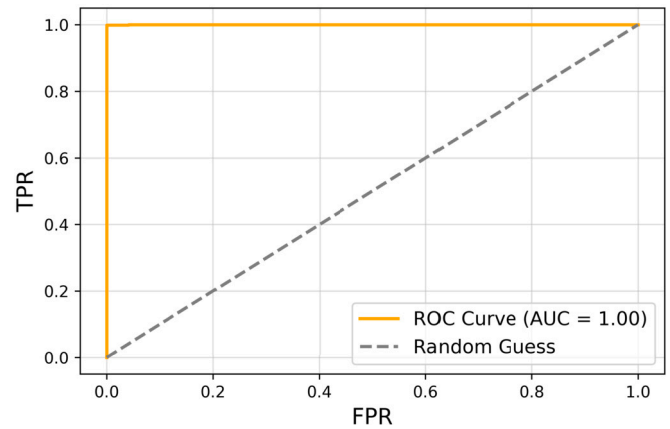


Fig. 10. The ROC curve for RoBERTa on balanced data.

Table 10

The Positive and Negative Coefficients Using LIME for the Example-1.

Positive Coefficients		Negative Coefficients	
Word	Value	Word	Value
'Alert'	2.586283490431408	'for'	-0.4540554753154869
'10k'	2.5443085492464883	'Big'	-0.424707563134796
'voucher'	2.19896727844678	'U'	-0.3875706185927502
'150'	1.5489612217746545	'bar'	-0.2933243630343485
'selected'	1.5372338239788719	'The'	-0.17252983760850746
'Brother'	1.2084815552598014	'computer'	-0.019025118002313234
'has'	0.6819267212480585		
'or'	0.02814567596003713		

where “True Label” = 0 denotes ham and “True Label” = 1 denotes spam.

Table 11

The Positive and Negative Coefficients Using LIME for the Example-2.

Positive Coefficients		Negative Coefficients	
Word	Value	Word	Value
'FREE'	3.186586145122694	'Thanks'	-0.5401725013063734
'Karaoke'	1.5886755775421426	'the'	-0.3920060404493261
'Vote'	1.314393037077685	'just'	-0.28230176631326165
'link'	1.257098816616926		
'for'	1.0052090127637938		
'mobile'	0.8919310037119642		
'with'	0.597081408704299		
'SING'	0.5501241601784591		
'reply'	0.49702113599413067		
'stars'	0.3446891653595809		
'Now'	0.3248660570258343		
'sing'	0.2221033541586965		

Text with highlighted words

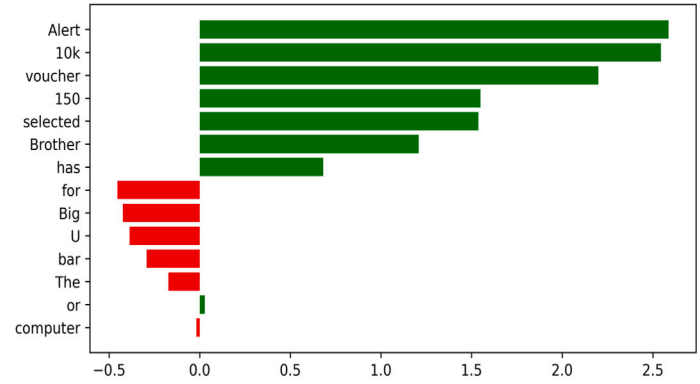
Big Brother Alert The computer has selected U for 10k bar or 150 voucher.

(a) Preprocessed SMS text with highlighted words (Example-1)

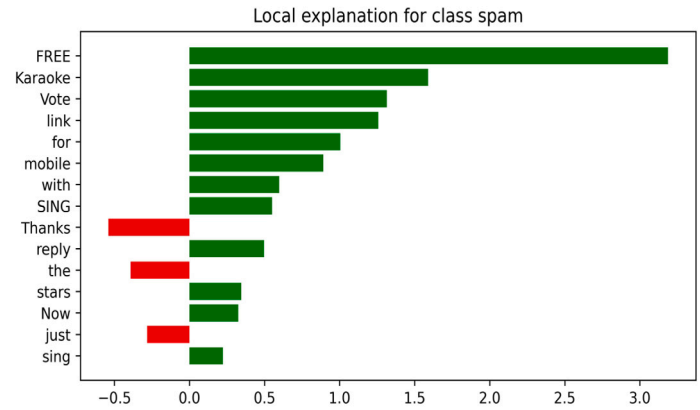
Text with highlighted words

Thanks for the Vote Now sing along with stars Karaoke on your mobile For a FREE link just reply SING now

(c) Preprocessed SMS text with highlighted words (Example-2)



(b) Explanation of highlighted words in spam text (Example-1)



(d) Explanation of highlighted words in spam text (Example-2)

Fig. 11. LIME Explanation for SMS spam messages.

Table 12

The Positive and Negative Coefficients Using LIME for the Example-3.

Positive Coefficients		Negative Coefficients	
Word	Value	Word	Value
'night'	0.080110152925621	'Good'	-0.022107989126383402
'care'	0.06923821968423888	'Take'	-0.020104893561268274
'amp'	0.060886333137515174		
'Sleepwell'	0.03722758230659266		
'my'	0.024641469910968265		
'dear'	0.018922945727309903		

Table 13

The Positive and Negative Coefficients Using LIME for the Example-4.

Positive Coefficients		Negative Coefficients	
Word	Value	Word	Value
'scream'	0.0628967823689254	'more'	-0.04109442935123623
'and'	0.05404817757012105	'Woohoo'	-0.03831983060032088
'weighed'	0.05100704096103593	'want'	-0.028705541788966896
'to'	0.04824724743333108	'lost'	-0.009499243825646576
'myself'	0.037156130726796105	'Omg'	-0.006713896998241079
'I'	0.02486380111242408		
'weight'	0.019592668979617558		

6. Discussion

The overall experiment was conducted using both imbalanced and balanced datasets to evaluate the performance of the proposed fine-tuned RoBERTa model. This dual approach allowed us to assess the model's effectiveness and robustness across varying data distributions. We employed a data-balancing technique for this model's effectiveness visu-

Table 14

The Positive and Negative Coefficients Using Transformer Interpret for the Example-1.

Positive Coefficients		Negative Coefficients	
Word	Value	Word	Value
'Big'	0.024297947664629425	'Brother'	-0.14376454412860262
'Alert'	0.20062465985674183	'The'	-0.20299253710630974
'selected'	0.3235594150691528	'computer'	-0.06358605308176257
'U'	0.14328277724673683	'has'	-0.09203341078407345
'10'	0.454875630595537	'for'	-0.013336465891418426
'k'	0.07143031221542949		
'bar'	0.16807094091950164		
'150'	0.6246831675560521		
'or'	0.11691662255361639		
'voucher'	0.29147154979015566		
'.'	0.023613104082805682		
'	0.17233550519021407		

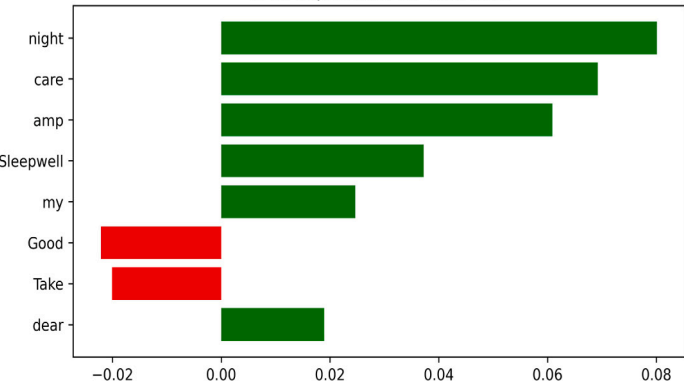
alization and balanced the data. On the imbalanced dataset, the model achieved 99.69% training and 99.37% testing accuracy with 0.0292 loss. On the other hand, our fine-tuned model achieved 99.95% training accuracy and 99.84% testing accuracy with 0.0122 loss on the balanced dataset. To compare with our model performance, we selected several traditional ML models and transformer-based models on both dataset types. Among those tested models, the RoBERTa model performed better. Furthermore, the optimized model performed well among the tested models and outperformed previous studies, demonstrating its effectiveness.

Along with the performance analysis of our model, we also worked with XAI techniques which can provide answers to various types of questions such as "Why did the model make a particular prediction?"

Text with highlighted words

Good night my dear Sleepwell amp Take care

(a) Preprocessed SMS text with highlighted words (Example-3)

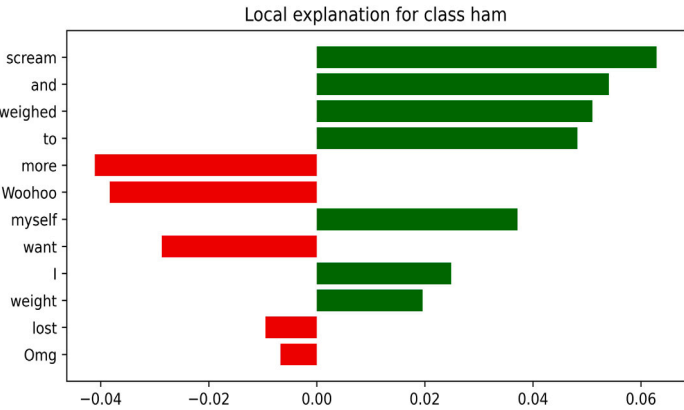


(b) Explanation of highlighted words in ham text (Example-3)

Text with highlighted words

Omg I want to scream weighed myself and lost more weight
Woohoo

(c) Preprocessed SMS text with highlighted words (Example-4)



(d) Explanation of highlighted words in ham text (Example-4)

Fig. 12. LIME Explanation for SMS ham messages.

Legend: ■ Negative □ Neutral ■ Positive					
True Label	Predicted Label	Attribution Label	Attribution Score	Word Importance	
1	LABEL_1 (1.00)	LABEL_1	2.10	#s Big Brother Alert The computer has selected U for 10 k bar or 150 voucher . #/s	

(a) Explanation for SMS spam text (Example-1)

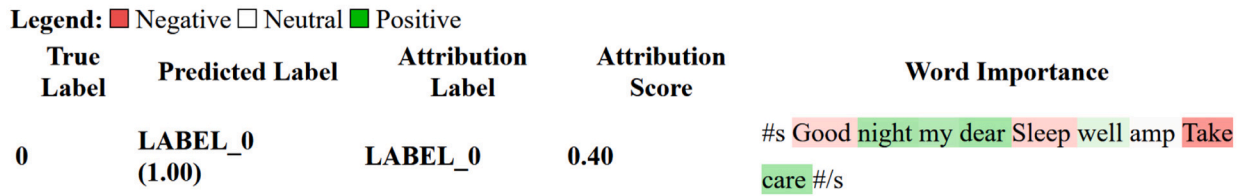
Legend: ■ Negative □ Neutral ■ Positive					
True Label	Predicted Label	Attribution Label	Attribution Score	Word Importance	
1	LABEL_1 (1.00)	LABEL_1	0.24	#s Thanks for the Vote Now sing along with stars Kara oke on your mobile For a FREE link just reply S ING now #/s	

(b) Explanation for SMS spam text (Example-2)

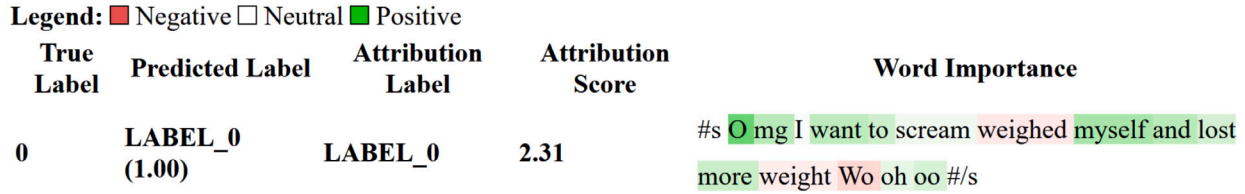
Fig. 13. Transformers Interpret Explanation for SMS spam messages.

and “Which features have the most significant impact on the model’s output?” related to the functioning and decision-making process of the model. For this, we used two well-known XAI techniques, such as LIME and Transformers Interpret to understand the models’ decision-making process. It is more difficult to understand how the RoBERTa model makes its predictions because, as we know, it is a transformer-based

model which is so complex in architecture. Feature importance analysis using XAI helps users to understand the factors that most influence a model’s decision-making process. By analyzing input features such as words or sub-words, XAI methods provide insights into the factors that contributed significantly to the specific classification of spam or ham messages. Mainly, these techniques calculated each word or sub-



(a) Explanation for SMS ham text (Example-3)



(b) Explanation for SMS ham text (Example-4)

Fig. 14. Transformers Interpret Explanation for SMS ham messages.

Table 15

The Positive and Negative Coefficients Using Transformers Interpret for the Example-2.

Positive Coefficients		Negative Coefficients	
Word	Value	Word	Value
'for'	0.18965947100466987	'Thanks'	-0.18213375681044702
'Vote'	0.04214511554119887	'the'	-0.16879623241117406
'stars'	0.050776681629629485	'sing'	-0.3819342785384402
'Kara'	0.13869346357463316	'along'	-0.32168706815694476
'on'	0.07859092995531514	'with'	-0.10687877218596162
'your'	0.2284359424426384	'just'	-0.15317418612821526
'For'	0.2290909616422906	'S'	-0.12666676012232256
'FREE'	0.2880400601164391	'now'	-0.031842205592812194
'link'	0.37679605294057267		
'reply'	0.04222149540549413		
'ING'	0.10210499982518297		
'mobile'	0.3437490059251579		
'a'	0.03278371376214245		

Table 16

The Positive and Negative Coefficients Using Transformers Interpret for the Example-3.

Positive Coefficients		Negative Coefficients	
Word	Value	Word	Value
'night'	0.3506601184212354	'Good'	-0.23430671367151
'my'	0.30588133708098775	'Sleep'	-0.2558005048167494
'dear'	0.3750717679647341	'Take'	-0.6100631898260275
'care'	0.36718414960882323	'amp'	-0.025046901303540856

word coefficient where some words indicate a positive coefficient and some negative coefficient, which indicates that these words with positive coefficients are considered more indicative to classify a class, while these words with negative coefficients are considered more indicative to do not support the predicted class. This overall process of explanation shows the transparency of our proposed model, by which users can gain insights into the prediction.

The experimental results demonstrate an enhancement of our proposed transformer-based model compared to other previously proposed approaches in SMS spam detection. LLMs in particular are being adopted quickly, which is driving ongoing innovation and growth in the field of natural language processing [7]. In our future work, we have a plan to explore several popular LLMs such as LLaMA, Gemini, Falcon, Mistral, etc. We will also collect more relevant datasets to explore our fine-

Table 17

The Positive and Negative Coefficients Using Transformers Interpret for the Example-4.

Positive Coefficients		Negative Coefficients	
Word	Value	Word	Value
'O'	0.6168087105189476	'weighed'	-0.14799138790010427
'mg'	0.285592216107348	'weight'	-0.1102382677942815
'I'	0.02889748690680515	'Wo'	-0.23351373922826477
'want'	0.2613656158757424		
'to'	0.21709997128226066		
'scream'	0.0764171095678442		
'myself'	0.3498253922866739		
'and'	0.2869778663987718		
'lost'	0.18199395807032306		
'more'	0.22200118513344438		
'oh'	0.11642180604530727		
'oo'	0.1630583357991135		

tuned model with explainability analysis. Although we worked with a data augmentation technique in this experiment, and it showed great impact on model performance, we intend to create synthetic data or sophisticated sampling strategies for managing unbalanced datasets in the future. Furthermore, studying models in depth to improve their robustness through explainable AI approaches is consistent with the continual goal of enhancing AI systems' trustworthiness and comprehension.

7. Conclusion

A significant improvement in SMS spam detection has been achieved by utilizing cutting-edge natural language processing algorithms. By leveraging LLMs, we can enhance the global computing experience and resolve long-standing societal issues. In this paper, we utilized modified transformer-based models to identify SMS spam. The entire investigation was conducted using a benchmark dataset, despite data imbalance issues. However, these challenges were effectively mitigated through data augmentation such as back translation. The experimental results showed that, compared to traditional models, our transformer-based model, more specifically RoBERTa performed better at differentiating between spam and valid SMSs. To achieve such high performance, the model is optimized and fine-tuned and trained on pre-processed real data. This improved version of RoBERTa achieved a testing accuracy of 99.84% in the SMS spam classification task. Moreover, explainable AI techniques such as LIME and Transformers Interpret are indeed used to see the explanations of the model prediction by calculating the

word coefficient. The explanation helps researchers and professionals understand how the model works and make decisions. By enhancing transparency improves the model's robustness and reliability, thereby increasing user confidence in its outcomes.

CRedit authorship contribution statement

Mohammad Amaz Uddin: Writing – original draft, Software, Methodology, Formal analysis, Data curation, Conceptualization, Validation, Visualization. **Muhammad Nazrul Islam:** Writing – review & editing. **Leandros Maglaras:** Writing – review & editing. **Helge Janicke:** Writing – review & editing. **Iqbal H. Sarker:** Writing – review & editing, Supervision, Conceptualization, Formal analysis, Methodology.

Declaration of competing interest

The authors declare no conflict of interests.

References

- [1] X. Liu, H. Lu, A. Nayak, A spam transformer model for sms spam detection, *IEEE Access* 9 (2021) 80253–80263.
- [2] C. Oswald, S.E. Simon, A. Bhattacharya, Spams: intention analysis-driven sms spam detection using bert embeddings, *ACM Trans. Web* 16 (3) (2022) 1–27.
- [3] S.M.M. Hossain, A. Sen, K. Deb, Detecting spam sms using self attention mechanism, in: *International Conference on Intelligent Computing & Optimization*, Springer, 2022, pp. 175–184.
- [4] F. Wei, T. Nguyen, A lightweight deep neural model for sms spam detection, in: *2020 International Symposium on Networks, Computers and Communications (ISNCC)*, IEEE, 2020, pp. 1–6.
- [5] I.H. Sarker, AI-Driven Cybersecurity and Threat Intelligence: Cyber Automation, Intelligent Decision-Making and Explainability, Springer Nature, 2024.
- [6] W.X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, et al., A survey of large language models, *arXiv preprint arXiv:2303.18223*.
- [7] I.H. Sarker, Llm potentiality and awareness: a position paper from the perspective of trustworthy and responsible ai modeling, *Discov. Artif. Intell.* 4 (1) (2024) 40.
- [8] Y. Yao, J. Duan, K. Xu, Y. Cai, E. Sun, Y. Zhang, A survey on large language model (llm) security and privacy: the good, the bad, and the ugly, *arXiv preprint arXiv:2312.02003*.
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [10] K. Han, A. Xiao, E. Wu, J. Guo, C. Xu, Y. Wang, Transformer in transformer, *Adv. Neural Inf. Process. Syst.* 34 (2021) 15908–15919.
- [11] S. Jamal, H. Wimmer, I. Sarker, An improved transformer-based model for detecting phishing, spam, and ham: a large language model approach, *arXiv preprint arXiv:2311.04913*.
- [12] M. Korotkev, Bert: a review of applications in natural language processing and understanding, *arXiv preprint arXiv:2103.11943*.
- [13] A.H. Mohammed, A.H. Ali, Survey of bert (bidirectional encoder representation transformer) types, *J. Phys. Conf. Ser.* 1963 (2021) 012173.
- [14] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: pre-training of deep bidirectional transformers for language understanding, *arXiv preprint arXiv:1810.04805*.
- [15] R. Anil, B. Ghazi, V. Gupta, R. Kumar, P. Manurangsi, Large-scale differentially private bert, *arXiv preprint arXiv:2108.01624*.
- [16] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, R. Soricut, Albert: a lite bert for self-supervised learning of language representations, *arXiv preprint arXiv:1909.11942*.
- [17] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, Roberta: a robustly optimized bert pretraining approach, *arXiv preprint arXiv:1907.11692*.
- [18] V. Sanh, L. Debut, J. Chaumond, T. Wolf, Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter, *arXiv preprint arXiv:1910.01108*.
- [19] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, Q. Liu, Tinybert: distilling bert for natural language understanding, *arXiv preprint arXiv:1909.10351*.
- [20] F. Xu, H. Uszkoreit, Y. Du, W. Fan, D. Zhao, J. Zhu, Explainable ai: a brief survey on history, research areas, approaches and challenges, in: *Natural Language Processing and Chinese Computing: 8th CCF International Conference, NLPCC 2019, Dunhuang, China, October 9–14, 2019, Proceedings*, Springer, 2019, pp. 563–574.
- [21] P. Linardatos, V. Papastefanopoulos, S. Kotsiantis, Explainable ai: a review of machine learning interpretability methods, *Entropy* 23 (1) (2020) 18.
- [22] I.H. Sarker, H. Janicke, A. Mohsin, A. Gill, L. Maglaras, Explainable ai for cybersecurity automation, intelligence and trustworthiness in digital twin: methods, taxonomy, challenges and prospects, *ICT Express*.
- [23] M.A. Abid, S. Ullah, M.A. Siddique, M.F. Mushtaq, W. Aljedaani, F. Rustam, Spam sms filtering based on text features and supervised machine learning techniques, *Multimed. Tools Appl.* 81 (28) (2022) 39853–39871.
- [24] S. Mishra, D. Soni, Dsmishms-a system to detect smishing sms, *Neural Comput. Appl.* 35 (7) (2023) 4975–4992.
- [25] G. Sonowal, Detecting phishing sms based on multiple correlation algorithms, *SN Comput. Sci.* 1 (6) (2020) 361.
- [26] S.M. Nagare, P.P. Dapke, S.A. Quadri, S.B. Bandal, M.R. Baheti, Short message service (sms) mobile spam detection using naive Bayes, in: *2024 5th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI)*, IEEE, 2024, pp. 67–70.
- [27] S. Mishra, D. Soni, Smishing detector: a security model to detect smishing through sms content analysis and url behavior analysis, *Future Gener. Comput. Syst.* 108 (2020) 803–815.
- [28] J.W. Joo, S.Y. Moon, S. Singh, J.H. Park, S-detector: an enhanced security model for detecting smishing attack for mobile computing, *Telecommun. Syst.* 66 (2017) 29–38.
- [29] T. Xia, X. Chen, A discrete hidden Markov model for sms spam detection, *Appl. Sci.* 10 (14) (2020) 5011.
- [30] A. Hourabi, M.A. Mahmood, Q.M. Alzubi, A hybrid cnn-lstm model for sms spam detection in Arabic and English messages, *Future Internet* 12 (9) (2020) 156.
- [31] P.K. Roy, J.P. Singh, S. Banerjee, Deep learning to filter sms spam, *Future Gener. Comput. Syst.* 102 (2020) 524–533.
- [32] O. Abayomi-Alli, S. Misra, A. Abayomi-Alli, A deep learning method for automatic sms spam classification: performance of learning algorithms on indigenous dataset, *Concurr. Comput., Pract. Exp.* 34 (17) (2022) e6989.
- [33] T. Sahnoud, D.M. Mikki, Spam detection using bert, *arXiv preprint arXiv:2206.02443*.
- [34] V.S. Tida, S. Hsu, Universal spam detection using transfer learning of bert model, *arXiv preprint arXiv:2202.03480*.
- [35] T. Chen, C. Guestrin, Xgboost: a scalable tree boosting system, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [36] L. Wang, *Support Vector Machines: Theory and Applications*, vol. 177, Springer Science & Business Media, 2005.
- [37] G. Guo, H. Wang, D. Bell, Y. Bi, K. Greer, Knn model-based approach in classification, in: *On the Move to Meaningful Internet Systems 2003: CoopIS, DOA, and ODBASE: OTM Confederated International Conferences, CoopIS, DOA, and ODBASE 2003, Catania, Sicily, Italy, November 3–7, 2003. Proceedings*, Springer, 2003, pp. 986–996.
- [38] L. Breiman, Random forests, *Mach. Learn.* 45 (2001) 5–32.
- [39] A.F. Adoma, N.-M. Henry, W. Chen, Comparative analyses of bert, roberta, distilbert, and xlnet for text-based emotion recognition, in: *2020 17th International Computer Conference on Wavelet Active Media Technology and Information Processing (IC-CWAMTIP)*, IEEE, 2020, pp. 117–121.
- [40] K.L. Tan, C.P. Lee, K.S.M. Anbananthen, K.M. Lim, Roberta-lstm: a hybrid model for sentiment analysis with transformer and recurrent neural network, *IEEE Access* 10 (2022) 21517–21525.
- [41] P.M. Radiuk, Impact of training set batch size on the performance of convolutional neural networks for diverse datasets, *Inf. Technol. Manag. Sci.* 20 (1) (2017) 20–24.
- [42] R. Lin, Analysis on the selection of the appropriate batch size in cnn neural network, in: *2022 International Conference on Machine Learning and Knowledge Engineering (MLKE)*, IEEE, 2022, pp. 106–109.
- [43] B. Schmeiser, Batch size effects in the analysis of simulation output, *Oper. Res.* 30 (3) (1982) 556–568.
- [44] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, *arXiv preprint arXiv:1711.05101*.
- [45] Z. Zhuang, M. Liu, A. Cutkosky, F. Orabona, Understanding adamw through proximal methods and scale-freeness, *arXiv preprint arXiv:2202.00089*.
- [46] Y. Zhang, K. Song, Y. Sun, S. Tan, M. Udell, “Why should you trust my explanation?” Understanding uncertainty in lime explanations, *arXiv preprint arXiv:1904.12991*.
- [47] A. Holzinger, A. Saranti, C. Molnar, P. Biecek, W. Samek, Explainable ai methods: a brief overview, in: *International Workshop on Extending Explainable AI Beyond Deep Models and Classifiers*, Springer, 2022, pp. 13–38.
- [48] R. Dwivedi, D. Dave, H. Naik, S. Singhal, R. Omer, P. Patel, B. Qian, Z. Wen, T. Shah, G. Morgan, et al., Explainable ai (xai): core ideas, techniques, and solutions, *ACM Comput. Surv.* 55 (9) (2023) 1–33.
- [49] M.T. Ribeiro, S. Singh, C. Guestrin, “Why should I trust you?” Explaining the predictions of any classifier, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.
- [50] C. Molnar, *Interpretable Machine Learning*, Lulu.com, 2020.